



EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E WEB PROGRAMMING AND INTERFACE

HAPI.JS DOCUMENTATION

- Resources:
 - Official documentation: [Hapi.js Docs](#)
 - Key sections:
 - Getting Started
 - Routing
 - Validation
 - Error Handling

HAPI.JS DOCUMENTATION

- Importance of Documentation:
 - Guides best practices
 - Helps in leveraging framework features

TESTING HAPI.JS ROUTES

- Testing Frameworks:
 - Jest
 - Mocha & Chai

TESTING MONGODB QUERIES

- Testing Framework:
 - Use Jest or Mocha

TESTING FRAMEWORKS

- Jest
 - A popular testing framework with built-in assertion library.
 - Supports mock functions and is easy to configure.

TESTING FRAMEWORKS

- Mocha & Chai
 - Mocha: A flexible testing framework for Node.js.
 - Chai: An assertion library that works with Mocha, allowing for BDD (Behavior Driven Development) style testing.

SETTING UP JEST

- Installation:

```
npm install --save-dev jest @types/jest supertest
```

- Add test script in package.json:

```
"scripts": {  
  "test": "jest"  
}
```


SAMPLE CODE USING JEST

- Here's how you can set up and run tests using Jest for Hapi.js routes.
- Example: Testing a Hapi.js Route
- Test File: `server.test.js`
- Run the Tests: `npm test`

TESTING MONGODB QUERIES

- Setting Up Mocha & Chai
- Installation:

```
npm install --save-dev mocha chai mongoose
```

- Add test script in package.json:

```
"scripts": {  
  "test": "mocha"  
}
```

SAMPLE CODE USING MOCHA & CHAI

- Example: Testing a MongoDB Query

```
// models/User.js
```

```
const mongoose = require('mongoose');  
const UserSchema = new mongoose.Schema({  
  name: String,  
  age: Number  
});  
module.exports = mongoose.model('User',  
UserSchema);
```

SAMPLE CODE USING MOCHA & CHAI

- Example: Testing a MongoDB Query

```
// models/User.js
```

```
const mongoose = require('mongoose');  
const UserSchema = new mongoose.Schema({  
  name: String,  
  age: Number  
});  
module.exports = mongoose.model('User',  
UserSchema);
```

API DOCUMENTATION TEMPLATE

- Authentication
- Type: [e.g., Bearer Token, API Key]
- Authentication Method: [Description of how to authenticate]
- Example:

Authorization: Bearer YOUR_TOKEN_HERE

ENDPOINTS

1. [Endpoint Name]

URL: [GET/POST/PUT/DELETE] /path/to/endpoint

Description: [Description of what this endpoint does.]

Request Parameters:

Query Parameters:

param1 (type, required/optional):
[Description]

param2 (type, required/optional):
[Description]

ENDPOINTS

Path Parameters:

param1 (type, required/optional):
[Description]

Request Body:

```
{  
  "field1": "value",  
  "field2": "value"  
}
```

ENDPOINTS

Response:

Success Response:

Code: 200 OK

Content:

```
{  
  "field1": "value",  
  "field2": "value"  
}
```


ENDPOINTS

Error Response:

Code: 400 Bad Request

Content:

```
{  
  "error": "Description of the error"  
}
```

ERROR CODES

Code	Message	Description
200	OK	The request was successful.
400	Bad Request	The request was invalid.
401	Unauthorized	Authentication failed.
404	Not Found	The requested resource was not found.
500	Internal Server Error	An unexpected error occurred.

RATE LIMITING

- Limit: [e.g., 100 requests per hour]
- Response Headers:
 - X-RateLimit-Limit: The maximum number of requests allowed.
 - X-RateLimit-Remaining: The number of requests remaining in the current window.
 - X-RateLimit-Reset: The time when the rate limit resets.

CONTACT INFORMATION

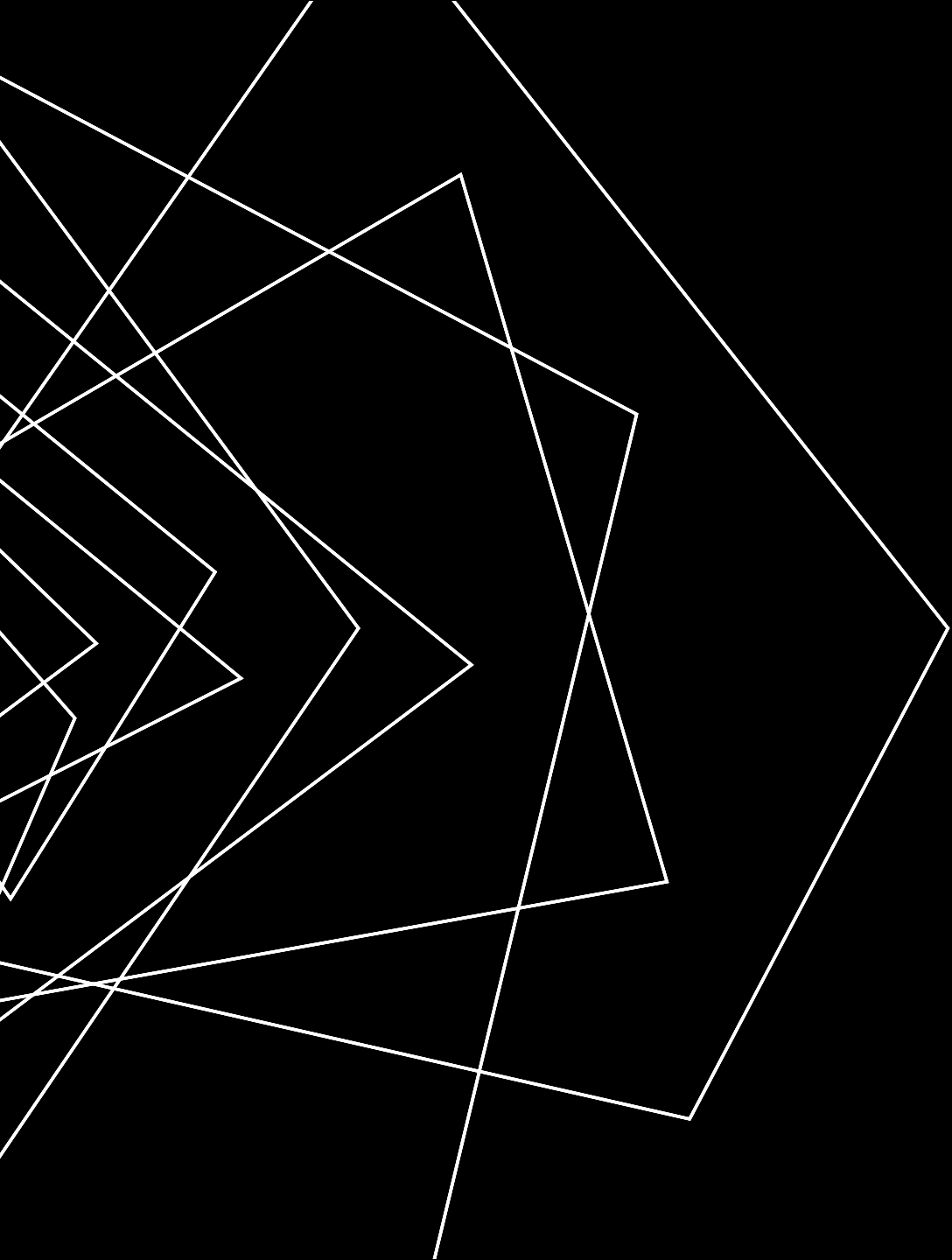
- Support Email: [support@yourservice.com]
- Documentation URL: [Link to complete documentation]

CONCLUSION

- Hapi.js provides a robust framework for building applications.
- Testing routes and MongoDB queries is essential for maintaining code quality.

CONCLUSION

- Effectively test Hapi.js routes
- Effectively test MongoDB queries
- Ensuring that your application behaves as expected and is robust against changes.
- Make sure to run your tests regularly during development!



THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335